

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Object Orientation

Overloading , overriding , encapsulation, constructor, polymorphism, is-a , has-a relations

Total Questions : 66

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Object Orientation

Questions Question - 1

What is the output for the below code ?

```
1. public class Test{
2.     public static void main (String[] args) {
3.         Test t1 = new Test();
4.         Test t2 = new Test();
5.         if(t1.equals(t2)){
6.             System.out.println("equal");
7.         }else{
8.             System.out.println("Not equal");
9.         }
10.    }
11. }
```

1. Not equal
2. equal
3. Compilation fails due to an error on line 5 - equals method not defined in Test class
4. Compilation fails due to an error on lines 3 and 4

Explanation :

A is the correct answer.

Every class in Java is a subclass of class Object, and Object class has equals method. In Test class equals method is not implemented. equals method of Object class only check for reference so return false in this case. t1.equals(t1) will return true.

Question - 2

What is the output for the below code ?

```
1. public class Test{
2.     public static void main (String[] args) {
3.         Test t1 = new Test();
4.         Test t2 = new Test();
5.         if(t1 instanceof Object){
6.             System.out.println("equal");
7.         }else{
8.             System.out.println("Not equal");
9.         }
10.    }
11. }
```

1. Not equal
2. equal
3. Compilation fails due to an error on line 5
4. Compilation fails due to an error on lines 3 and 4

Explanation :

B is the correct answer.

Every class in Java is a subclass of class Object, so every object is instance of Object.

Question - 3

What is the output for the below code ?

```
public class A {  
    public void printValue() {  
        System.out.println("Value-A");  
    }  
}  
  
public class B extends A {  
    public void printName() {  
        System.out.println("Name-B");  
    }  
}  
  
1. public class Test {  
2.     public static void main (String[] args) {  
3.         B b = new B();  
4.         b.printName();  
5.         b.printValue();  
6.     }  
7. }
```

1. Value-A Name-B
2. Name-B Value-A
3. Compilation fails due to an error on line 5
4. Compilation fails due to an error on lines 4

Explanation :

B is the correct answer.

Class B extended Class A therefore all methods of Class A will be available to class B except private methods.

Question - 4

What is the output for the below code ?

```
public class A {
```

```

        public void printValue(){
            System.out.println("Value-A");
        }
    }

    public class B extends A{
        public void printNameB(){
            System.out.println("Name-B");
        }
    }

    public class C extends A{

        public void printNameC(){
            System.out.println("Name-C");
        }

    }

1. public class Test{
2.     public static void main (String[] args) {
3.         B b = new B();
4.         C c = new C();
5.         newPrint(b);
6.         newPrint(c);
7.     }
8.     public static void newPrint(A a){
9.         a.printValue();
10.    }
11. }

```

- 1.Value-A Name-B
- 2.Value-A Value-A
- 3.Value-A Name-C
- 4.Name-B Name-C

Explanation :

B is the correct answer.

Class B extended Class A therefore all methods of Class A will be available to class B except private methods. Class C extended Class A therefore all methods of Class A will be available to class C except private methods.

Question - 5

What is the output for the below code ?

```

    public class A1 {
        public void printName(){
            System.out.println("Value-A");
        }
    }

    public class B1 extends A1{
        public void printName(){

```

```

        System.out.println("Name-B");
    }
}

public class Test{
    public static void main (String[] args) {
        A1 b = new B1();
        newPrint(b);
    }
    public static void newPrint(A1 a){
        a.printName();
    }
}

```

- 1.Value-A
- 2.Name-B
- 3.Value-A Name-B
- 4.Compilation fails

Explanation :

B is the correct answer.

This is an example of polymorphism. When someone has an A1 reference that NOT refers to an A1 instance, but refers to an A1 subclass instance, the caller should be able to invoke printName() on the A1 reference, but the actual runtime object (B1 instance) will run its own specific printName() method. So printName() method of B1 class executed.

Question - 6

What is the output for the below code ?

```

public class A {
    public void printName(){
        System.out.println("Value-A");
    }
}

public class B extends A{
    public void printName(){
        System.out.println("Name-B");
    }
}

public class C extends A{
    public void printName(){
        System.out.println("Name-C");
    }
}

```

1. public class Test{
2. public static void main (String[] args) {

```

3.      A b = new B();
4.      C c = new C();
5.      b = c;
6.      newPrint(b);
7.  }
8.  public static void newPrint(A a){
9.      a.printName();
10. }
11. }

```

1.Name-B

2.Name-C

3.Compilation fails due to an error on lines 5

4.Compilation fails due to an error on lines 9

Explanation :

B is the correct answer.

Reference variable can refer to any object of the same type as the declared reference OR can refer to any subtype of the declared type. Reference variable "b" is type of class A and reference variable "c" is a type of class C but reference variable "c" is a subtype of A class. So it works properly.

Question - 7

What is the output for the below code ?

```

public class A {
    public void printName() {
        System.out.println("Value-A");
    }
}

public class B extends A{
    public void printName() {
        System.out.println("Name-B");
    }
}

public class C extends A{

    public void printName() {
        System.out.println("Name-C");
    }

}

```

```

1. public class Test{
2.     public static void main (String[] args) {
3.         B b = new B();
4.         C c = new C();
5.         b = c;
6.         newPrint(b);
7.     }
8.     public static void newPrint(A a){

```

```
9.      a.printName();
10. }
11. }
```

- 1.Name-B
- 2.Name-C
- 3.Compilation fails due to an error on lines 5
- 4.Compilation fails due to an error on lines 9

Explanation :

C is the correct answer.

Reference variable can refer to any object of the same type as the declared reference OR can refer to any subtype of the declared type. Reference variable "b" is type of class B and reference variable "c" is a type of class C. So Compilation fails.

Question - 8

What is the output for the below code ?

```
public class C {
}

public class D extends C{
}

public class A {
    public C getOBJ(){
        System.out.println("class A - return C");
        return new C();
    }
}

public class B extends A{
    public D getOBJ(){
        System.out.println("class B - return D");
        return new D();
    }
}

public class Test {

    public static void main(String... args) {
        A a = new B();
        a.getOBJ();
    }
}
```

}

- 1.class A - return C
- 2.class B - return D
- 3.Compilation fails
- 4.Compilation succeed but no output

Explanation :

B is the correct answer.

From J2SE 5.0 onwards, return type in the overriding method can be same or subtype of the declared return type of the overridden (superclass) method.

Question - 9

What is the output for the below code ?

```
public class B {  
  
    public String getCountryName() {  
        return "USA";  
    }  
  
    public StringBuffer getCountryName() {  
        StringBuffer sb = new StringBuffer();  
        sb.append("UK");  
        return sb;  
    }  
  
    public static void main(String[] args) {  
        B b = new B();  
        System.out.println(b.getCountryName().toString());  
    }  
}
```

- 1.USA
- 2.Compile with error
- 3.UK
- 4.Compilation succeed but no output

Explanation :

B is the correct answer.

Compile with error : You cannot have two methods in the same class with signatures that only differ by return type.

Question - 10

What is the output for the below code ?

```
public class A1 {
    public void printName(){
        System.out.println("Value-A");
    }
}

public class B1 extends A1{
    protected void printName(){
        System.out.println("Name-B");
    }
}

public class Test{
    public static void main (String[] args) {
        A1 b = new B1();
        newPrint(b);
    }
    public static void newPrint(A1 a){
        a.printName();
    }
}
```

- 1.Value-A
- 2.Name-B
- 3.Value-A Name-B
- 4.Compilation fails

Explanation :

D is the correct answer.

The access level can not be more restrictive than the overridden method's.
Compilation fails because of protected method name printName in class B1.

Question - 11

What is the output for the below code ?

```
public class A {
    private void printName(){
        System.out.println("Value-A");
    }
}

public class B extends A{
    public void printName(){
        System.out.println("Name-B");
    }
}

public class Test{
    public static void main (String[] args) {
        B b = new B();
    }
}
```



```

        b.printName();
    }
}

```

- 1.Value-A
- 2.Name-B
- 3.Value-A Name-B
- 4.Compilation fails - private methods can't be override

Explanation :

B is the correct answer.

You can not override private method , private method is not available in subclass . In this case printName() method a class A is not overriding by printName() method of class B. printName() method of class B different method. So you can call printName() method of class B.

Question - 12

What is the output for the below code ?

```

public class A {
    private void printName() {
        System.out.println("Value-A");
    }
}

public class B extends A{
    public void printName() {
        System.out.println("Name-B");
    }
}

```

```

1. public class Test{
2.     public static void main (String[] args) {
3.         A b = new B();
4.         b.printName();
5.     }
6. }

```

- 1.Value-A
- 2.Compilation fails due to an error on lines 4
- 3.Name-B
- 4.Compilation fails - private methods can't be override

Explanation :

B is the correct answer.

You can not override private method , private method is not available in subclass . printName() method in class A is private so compiler complain about b.printName() . The method printName() from the type A is not visible.

Question - 13

What is the output for the below code ?

```
import java.io.FileNotFoundException;

public class A {
    public void printName() throws FileNotFoundException {
        System.out.println("Value-A");
    }
}

public class B extends A{
    public void printName() throws Exception{
        System.out.println("Name-B");
    }
}

public class Test{
    public static void main (String[] args) throws Exception{
        A a = new B();
        a.printName();
    }
}
```

- 1.Value-A
- 2.Compilation fails-Exception Exception is not compatible with throws clause in A.printName()
- 3.Name-B
- 4.Compilation succeed but no output

Explanation :

B is the correct answer.

Subclass overriding method must throw checked exceptions that are same or subclass of the exception thrown by superclass method.

Question - 14

What is the output for the below code ?

```
import java.io.FileNotFoundException;

public class A {
    public void printName() throws FileNotFoundException {
        System.out.println("Value-A");
    }
}

public class B extends A{
    public void printName() throws NullPointerException{
```

```

        System.out.println("Name-B");
    }
}

public class Test{
    public static void main (String[] args) throws Exception{
        A a = new B();
        a.printName();
    }
}

```

- 1.Value-A
- 2.Compilation fails-Exception NullPointerException is not compatible with throws clause in A.printName()
- 3.Name-B
- 4.Compilation succeed but no output

Explanation :

C is the correct answer.

The overriding method can throw any unchecked (runtime) exception, regardless of exception thrown by overridden method. NullPointerException is RuntimeException so compiler not complain.

Question - 15

What is the output for the below code ?

```

public class A {
    public static void printName() {
        System.out.println("Value-A");
    }
}

public class B extends A{
    public void printName(){
        System.out.println("Name-B");
    }
}

public class Test{
    public static void main (String[] args) throws Exception{
        A a = new B();
        a.printName();
    }
}

```

- 1.Value-A
- 2.Compilation fails-This instance method cannot override the static method from A

3.Name-B

4.Compilation succeed but no output

Explanation :

B is the correct answer.

You cannot override a static method . Compilation fails-This instance method cannot override the static method from A.

Question - 16

What is the output for the below code ?

```
public class A {  
    public A(){  
        System.out.println("A");  
    }  
    public A(int i){  
        this();  
        System.out.println(i);  
    }  
}
```

```
public class B extends A{  
    public B (){  
        System.out.println("B");  
    }  
    public B (int i){  
        this();  
        System.out.println(i+3);  
    }  
}
```

```
public class Test{  
    public static void main (String[] args){  
        new B(5);  
    }  
}
```

1.A B 8

2.A 5 B 8

3.A B 5

4.B 8 A 5

Explanation :

A is the correct answer.

Constructor of class B call their superclass constructor of class A (public A()) , which execute first, and that constructors can be overloaded. Then come to constructor of class B (public B (int i)).

Question - 17

What is the output for the below code ?

```
public class A {
    public A(){
        System.out.println("A");
    }
    public A(int i){
        System.out.println(i);
    }
}

public class B extends A{
    public B (){
        System.out.println("B");
    }
    public B (int i){
        this();
        System.out.println(i+3);
    }
}

public class Test{

    public static void main (String[] args){
        new B(5);
    }
}
```

- 1.A B 8
- 2.A 5 B 8
- 3.A B 5
- 4.B 8 A 5

Explanation :

A is the correct answer.

Constructor of class B call their superclass constructor of class A (public A()) , which execute first, and that constructors can be overloaded. Then come to constructor of class B (public B (int i)).

Question - 18

What is the output for the below code ?

```
public class A {
    public final void printName() {
        System.out.println("Value-A");
    }
}
```

```

public class B extends A{
    public void printName(){
        System.out.println("Name-B");
    }
}

public class Test{
    public static void main (String[] args) throws Exception{
        A a = new B();
        a.printName();
    }
}

```

- 1.Value-A
- 2.Compilation fails-Cannot override the final method from A
- 3.Name-B
- 4.Compilation succeed but no output

Explanation :

B is the correct answer.

You cannot override a final method. Compilation fails-Cannot override the final method from A.

Question - 19

What is the output for the below code ?

```

public class A {
    public void printName() {
        System.out.println("Value-A");
    }
}

public class B extends A{
    public void printName(){
        System.out.println("Name-B");
        super.printName();
    }
}

public class Test{
    public static void main (String[] args) throws Exception{
        A a = new B();
        a.printName();
    }
}

```

- 1.Value-A
- 2.Name-B Value-A
- 3.Name-B
- 4.Value-A Name-B

Explanation :

B is the correct answer.

super.printName(); calls printName() method of class A.

Question - 20

What is the output for the below code ?

```
public class A {
    public void printName() {
        System.out.println("Name-A");
    }
}

public class B extends A{
    public void printName() {
        System.out.println("Name-B");
    }

    public void printValue() {
        System.out.println("Value-B");
    }
}
```

```
1. public class Test{
2.     public static void main (String[] args){
3.         A b = new B();
4.         b.printValue();
5.     }
6. }
```

- 1.Value-B
- 2.Compilation fails due to an error on lines 4
- 3.Name-B
- 4.Value-B Name-B

Explanation :

B is the correct answer.

You are trying to use that A reference variable to invoke a method that only class B has. The method printValue() is undefined for the type A. So compiler complain.

Question - 21

What is the output for the below code ?

```
public class A {
    public void printName() {
        System.out.println("Name-A");
    }
}

public class B extends A{
    public void printName() {
        System.out.println("Name-B");
    }

    public void printValue() {
        System.out.println("Value-B");
    }
}
```

```
1. public class Test{
2.     public static void main (String[] args){
3.         A a = new A();
4.         B b = (B)a;
5.         b.printName();
6.     }
7. }
```

1. Value-B
2. Compilation fails due to an error on lines 4
3. Name-B
4. Compilation succeed but Runtime Exception

Explanation :

D is the correct answer.

java.lang.ClassCastException: A cannot be cast to B . You can cast subclass to superclass but not superclass to subclass.upcast is ok. You can do B b = new B(); A a = (A)b;

Question - 22

What is the output for the below code ?

```
public class A {
    public void printName() {
        System.out.println("Name-A");
    }
}
```



```

    }

    public class B extends A{
        public void printName(){
            System.out.println("Name-B");
        }

        public void printValue() {
            System.out.println("Value-B");
        }
    }
}

```

```

1. public class Test{
2.     public static void main (String[] args){
3.         B b = new B();
4.         A a = (A)b;
5.         a.printName();
6.     }
7. }

```

1. Value-B
2. Compilation fails due to an error on lines 4
3. Name-B
4. Compilation succeed but Runtime Exception

Explanation :

C is the correct answer.

You can cast subclass to superclass but not superclass to subclass. upcast is ok.
Compile clean and output Name-B.

Question - 23

Which of the following statement is true ?

1. A class can implement more than one interface.
2. An interface can extend another interface.
3. All variables defined in an interface implicitly public, static, and final.
4. All of the above statements are true

Explanation :

D is the correct answer.

All of the above statements are true.

Question - 24

What is the output for the below code ?

```
1. public interface InfA {  
2.     protected String getName();  
3. }  
  
public class Test implements InfA{  
    public String getName(){  
        return "test-name";  
    }  
  
    public static void main (String[] args){  
        Test t = new Test();  
        System.out.println(t.getName());  
    }  
}
```

- 1.test-name
- 2.Compilation fails due to an error on lines 2
- 3.Compilation fails due to an error on lines 1
- 4.Compilation succeed but Runtime Exception

Explanation :

B is the correct answer.

Illegal modifier for the interface method InfA.getName(); only public and abstract are permitted

Question - 25

What is the output for the below code ?

```
public class A {  
    public void printName() {  
        System.out.println("Name-A");  
    }  
}  
  
1. public class B extends A{  
2.     public String printName(){  
3.         System.out.println("Name-B");  
4.         return null;  
5.     }  
6. }  
  
public class Test{  
    public static void main (String[] args){  
        A a = new B();  
        a.printName();  
    }  
}
```

- 1.Name-B

- 2.Compilation fails due to an error on lines 2
- 3.Name-A
- 4.Compilation fails due to an error on lines 4

Explanation :

B is the correct answer.

printName() method of class A and printName() method of class B has different return type. So printName() method of class B does not overriding printName() method of class A. But class B extends A so printName() method of class A is available in class B, So compiler complain about the same method name. Method overloading does not consider return types.

Question - 26

What is the output for the below code ?

```
public class D {
    int i;
    int j;
    public D(int i,int j){
        this.i=i;
        this.j=j;
    }

    public void printName() {
        System.out.println("Name-D");
    }
}

1. public class Test{
2.     public static void main (String[] args){
3.         D d = new D();
4.         d.printName();
5.
6.     }
7. }
```

- 1.Name-D
- 2.Compilation fails due to an error on lines 3
- 3.Compilation fails due to an error on lines 4
- 4.Compilation succeed but no output

Explanation :

B is the correct answer.

Since there is already a constructor in this class (public D(int i,int j)), the compiler won't supply a default constructor. If you want a no-argument constructor to overload the with-arguments version you already have, you have to define it by yourself. The constructor D() is undefined in class D. If you define explicit constructor then default constructor will not be available. You have to define explicitly like public D(){ } then

the above code will work. If no constructor into your class , a default constructor will be automatically generated by the compiler.

Question - 27

What is the output for the below code ?

```
public class D {
    int i;
    int j;
    public D(int i,int j){
        this.i=i;
        this.j=j;
    }

    public void printName() {
        System.out.println("Name-D");
    }

    public D(){

    }
}

1. public class Test{
2.     public static void main (String[] args){
3.         D d = new D();
4.         d.printName();
5.
6.     }
7. }
```

- 1.Name-D
- 2.Compilation fails due to an error on lines 3
- 3.Compilation fails due to an error on lines 4
- 4.Compilation succeed but no output

Explanation :

A is the correct answer.

The constructor D() is defined in class D. If you define explicit constructor then default constructor will not be available. You have to define explicitly like public D() {} .

Question - 28

Which of the following statement is true about constructor?

- 1.The constructor name must match the name of the class.
- 2.Constructors must not have a return type.

- 3.The default constructor is always a no arguments constructor.
- 4.All of the above statements are true

Explanation :

D is the correct answer.

All of the above statements are true.

Question - 29

What is the output for the below code ?

```
1. public class A {
2.     int i;
3.     public A(int i){
4.         this.i=i;
5.     }
6.     public void printName() {
7.         System.out.println("Name-A");
8.     }
9. }

10. public class C extends A{
11. }

12. public class Test{
13.     public static void main (String[] args){
14.         A c = new C();
15.         c.printName();
16.     }
17. }
```

- 1.Name-A
- 2.Compilation fails due to an error on lines 10
- 3.Compilation fails due to an error on lines 14
- 4.Compilation fails due to an error on lines 15

Explanation :

B is the correct answer.

Implicit super constructor A() is undefined for default constructor. Must define an explicit constructor. Every constructor invokes the constructor of its superclass implicitly. In this case constructor of class A is overloaded. So need to call explicitly from subclass constructor. You have to call superclass constructor explicitly . public class C extends A{ public C(){ super(7); } }

Question - 30

What is the output for the below code ?

```
public class A {
```

```

        public A(){
            System.out.println("A");
        }
    }

    public class B extends A{
        public B(){
            System.out.println("B");
        }
    }

    public class C extends B{
        public C(){
            System.out.println("C");
        }
    }

    public class Test{
        public static void main (String[] args){
            C c = new C();
        }
    }

```

- 1.A B C
- 2.C B A
- 3.C A B
- 4.Compilation fails

Explanation :

A is the correct answer.

Every constructor invokes the constructor of its superclass implicitly. Superclass constructor executes first then subclass constructor.

Question - 31

What is the output for the below code ?

```

public class A {

    public A(int i){
        System.out.println(i);
    }

}

1. public class B extends A{
2.     public B(){
3.         super(6);
4.         this();
5.     }
6. }

public class Test{
    public static void main (String[] args){

```

```

        B b = new B();
    }
}

```

- 1.6
- 2.0
- 3.Compilation fails due to an error on lines 3
- 4.Compilation fails due to an error on lines 4

Explanation :

D is the correct answer.

A constructor can NOT have both super() and this(). Because each of those calls must be the first statement in a constructor, you can NOT use both in the same constructor.

Question - 32

What is the output for the below code ?

```

1. public class A {
2.     public A(){
3.         this(8);
4.     }
5.     public A(int i){
6.         this();
7.     }
8. }

```

```

public class Test{
    public static void main (String[] args){
        A a = new A();
    }
}

```

- 1.8
- 2.0
- 3.Compilation fails due to an error on lines 1
- 4.Compilation fails due to an error on lines 3

Explanation :

D is the correct answer.

This is Recursive constructor invocation from both the constructor so compiler complain.

Question - 33

What is the output for the below code ?

```

1. public class Test {
2.     static int count;
3.     public Test(){
4.         count = count +1 ;
5.     }
6.     public static void main(String argv[]){
7.         new Test();
8.         new Test();
9.         new Test();
10.        System.out.println(count);
11.    }
12. }

```

1.3

2.0

3.1

4.Compilation fails due to an error on lines 2

Explanation :

A is the correct answer.

static variable count set to zero when class Test is loaded. static variables are related to class not instance so count increases on every new Test();

Question - 34

```

public class A {
    public void test1(){
        System.out.println("test1");
    }
}

```

```

public class B extends A{
    public void test2(){
        System.out.println("test2");
    }
}

```

```

1. public class Test{
2.     public static void main (String[] args){
3.         A a = new A();
4.         A b = new B();
5.         B b1 = new B();
6.         // insert code here
7.     }
8. }

```

Which of the following , inserted at line 6, will compile and print test2?

1.((B)b).test2();

2.(B)b.test2();

3.b.test2();

4.a.test2();

Explanation :

A is the correct answer.

((B)b).test2(); is proper cast. test2() method is in class B so need to cast b then only test2() is accessible. (B)b.test2(); is not proper cast without the second set of parentheses, the compiler thinks it is an incomplete statement.

Question - 35

What is the output for the below code ?

```
public class A {
    static String str = "";
    protected A() {
        System.out.println(str + "A");
    }
}

public class B extends A {
    private B () {
        System.out.println(str + "B");
    }
}

1. public class Test extends A {
2.     private Test() {
3.         System.out.println(str + "Test");
4.     }
5.     public static void main (String[] args) {
6.         new Test();
7.         System.out.println(str);
8.     }
9. }
```

1. A Test
2. A B Test
3. Compilation fails due to an error on lines 6
4. Compilation fails due to an error on lines 2

Explanation :

A is the correct answer.

Test class extends class A and there is no attempt to create class B object so private constructor in class B is not called . private constructor in class Test is okay. You can create object from the class where private constructor present but you can't from outside of the class.

Question - 36

```

public class A{
    private void test1(){

        System.out.println("test1 A");
    }
}

public class B extends A{
    public void test1(){

        System.out.println("test1 B");
    }
}

public class outputtest{
    public static void main(String[] args){
        A b = new B();
        b.test1();
    }
}

```

What is the output?

- 1.test1 A
- 2.test1 B
- 3.Not compile because test1() method in class A is not visible.
- 4.None of the above.

Explanation :

C is the correct answer.

Not compile because test1() method in class A is not private so it is not visible.

Question - 37

```

public class A{
    private void test1(){

        System.out.println("test1 A");
    }
}

public class B extends A{
    public void test1(){

        System.out.println("test1 B");
    }
}

public class Test{
    public static void main(String[] args){
        B b = new B();
        b.test1();
    }
}

```

```
}  
}
```

What is the output?

- 1.test1 B
- 2.test1 A
- 3.Not compile because test1() method in class A is not visible
- 4.None of the above

Explanation :

A is the correct answer.

This is not related to superclass , B class object calls its own method so it compile and output normally.

Question - 38

```
public class A{  
    public void test1(){  
  
        System.out.println("test1 A");  
    }  
}  
  
public class B extends A{  
    public void test1(){  
  
        System.out.println("test1 B");  
    }  
}  
  
public class Test{  
    public static void main(String[] args){  
        A b = new B();  
        b.test1();  
    }  
}
```

What is the output?

- 1.test1 B
- 2.test1 A
- 3.Not compile because test1() method in class A is not visible
- 4.None of the above

Explanation :

A is the correct answer.

Superclass reference of subclass point to subclass method and superclass variables.

Question - 39

What is the output of the below code ?

```
class c1
{
public static void main(String a[])
{
c1 ob1=new c1();
Object ob2=ob1;
System.out.println(ob2 instanceof Object);
System.out.println(ob2 instanceof c1);
}
}
```

- 1.Prints true,false
- 2.Print false,true
- 3.Prints true,true
- 4.compile time error

Explanation :

C is the correct answer.

instanceof operator checks for instance related to class or not.

Question - 40

```
public class C {
    public C(){
    }
}

public class D extends C {
    public D(int i){
        System.out.println("tt");
    }
    public D(int i, int j){
        System.out.println("tt");
    }
}
```

Is C c = new D(); works ?

- 1.It compile without any error
- 2.It compile with error

- 3.Can't say
- 4.None of the above

Explanation :

B is the correct answer.

C c = new D(); NOT COMPILE because D don't have default constructor. If super class has different constructor other then default then in the sub class you can't use default constructor

Question - 41

```
public class C {  
    public C(){  
    }  
}  
  
public class D extends C {  
    public D(int i){  
        System.out.println("tt");  
    }  
    public D(int i, int j){  
        System.out.println("tt");  
    }  
}
```

Is C c = new D(8); works ?

- 1.It compile without any error
- 2.It compile with error
- 3.Can't say
- 4.None of the above

Explanation :

A is the correct answer.

C c = new D(8); COMPILE because D don't have default constructor. If super class has different constructor other then default then in the sub class you can't use default constructor

Question - 42

```
public class C {
```

```

        public C(int i){

        }

    }

    public class D extends C {

        public D(){

        }

    }

```

is this code compile without error?

- 1.yes, compile without error.
- 2.No, compile with error.
- 3.Runtime Exception
- 4.None of the above

Explanation :

B is the correct answer.

We need to invoke Explicitly super constructor();

Question - 43

```

public class SuperClass {
    public int doIt(){
        System.out.println("super doIt()");
        return 1;
    }
}

public class SubClass extends SuperClass{

    public int doIt()
    {
        System.out.println("subclas doIt()");
        return 0;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();

        sb.doIt();

    }
}

```

What is the output ?

- 1.subclas doIt()
- 2.super doIt()
- 3.Compile with error
- 4.None of the above

Explanation :

A is the correct answer.

method are reference to sub class and variables are reference to superclass.

Question - 44

```
public class SuperClass {
    public int doIt(){
        System.out.println("super doIt()");
        return 1;
    }
}

public class SubClass extends SuperClass{

    public long doIt()
    {
        System.out.println("subclas doIt()");
        return 0;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();

        sb.doIt();
    }
}
```

What is the output ?

- 1.subclas doIt()
- 2.super doIt()
- 3.Compile with error
- 4.None of the above

Explanation :

C is the correct answer.

The return type is incompatible with SuperClass.doIt(). Return type of the method doIt() should be int.

Question - 45

```
public class SuperClass {
    private int doIt(){
        System.out.println("super doIt()");
        return 1;
    }
}

public class SubClass extends SuperClass{

    public long doIt()
    {
        System.out.println("subclas doIt()");
        return 0;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();

        sb.doIt();

    }
}
```

What is the output ?

- 1.subclas doIt()
- 2.super doIt()
- 3.Compile with error
- 4.None of the above

Explanation :

C is the correct answer.

The method doIt() from the type SuperClass is not visible.

Question - 46

```
public class SuperClass {
    public final int doIt(){
        System.out.println("super doIt()");
        return 1;
    }
}

public class SubClass extends SuperClass{

    public long doIt()
```



```

    {
        System.out.println("subclas doIt()");
        return 0;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();

        sb.doIt();

    }
}

```

What is the output ?

- 1.subclas doIt()
- 2.super doIt()
- 3.Compile with error
- 4.None of the above

Explanation :

C is the correct answer.

Cannot override the final method from SuperClass

Question - 47

What will be the result of compiling the following code:

```

public class SuperClass {
    public int doIt(String str, Integer... data) throws Exception{
        String signature = "(String, Integer[])";
        System.out.println(str + " " + signature);
        return 1;
    }
}

public class SubClass extends SuperClass{

    public int doIt(String str, Integer... data)
    {
        String signature = "(String, Integer[])";
        System.out.println("Overridden: " + str + " " + signature);
        return 0;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();
        sb.doIt("hello", 3);
    }
}

```

- 1.Overridden: hello (String, Integer[])
- 2.hello (String, Integer[])
- 3.Compile time error.
- 4.None of the above

Explanation :

C is the correct answer.

Unhandled exception type Exception.

Question - 48

What will be the result of compiling the following code:

```
public class SuperClass {
    public int doIt(String str, Integer... data) throws
    ArrayIndexOutOfBoundsException{
        String signature = "(String, Integer[])";
        System.out.println(str + " " + signature);
        return 1;
    }
}

public class SubClass extends SuperClass{

    public int doIt(String str, Integer... data) throws Exception
    {
        String signature = "(String, Integer[])";
        System.out.println("Overridden: " + str + " " + signature);
        return 0;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();
        try{
            sb.doIt("hello", 3);
        }catch(Exception e){

        }

    }
}
```

- 1.Overridden: hello (String, Integer[])
- 2.hello (String, Integer[])
- 3.The code throws an Exception at Runtime.
- 4.Compile with error

Explanation :
D is the correct answer.

Exception Exception is not compatible with throws clause in SuperClass.doIt(String, Integer[]). The same exception or subclass of that exception is allowed.

Question - 49

```
public class SuperClass {
    public ArrayList doIt(){
        ArrayList lst = new ArrayList();
        System.out.println("super doIt()");
        return lst;
    }
}

public class SubClass extends SuperClass{

    public List doIt()
    {
        List lst = new ArrayList();
        System.out.println("subclas doIt()");
        return lst;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();

        sb.doIt();

    }
}
```

What is the output ?

- 1.subclas doIt()
- 2.super doIt()
- 3.Compile with error
- 4.None of the above

Explanation :
C is the correct answer.

The return type is incompatible with SuperClass.doIt(). subtype return is eligible in jdk 1.5 for overridden method but not super type return. super class method return ArrayList so subclass overridden method should return same or sub type of ArrayList.

Question - 50

```
public class SuperClass {
    public List doIt(){
        List lst = new ArrayList();
        System.out.println("super doIt()");
        return lst;
    }
}

public class SubClass extends SuperClass{

    public ArrayList doIt()
    {
        ArrayList lst = new ArrayList();
        System.out.println("subclas doIt()");
        return lst;
    }

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();

        sb.doIt();

    }
}
```

What is the output ?

- 1.subclas doIt()
- 2.super doIt()
- 3.Compile with error
- 4.None of the above

Explanation :

A is the correct answer.

subtype return is eligible in jdk 1.5 for overridden method but not super type return. super class method return List so subclass overridden method should return same or sub type of List.

Question - 51

```
class C{
int i;
public static void main (String[] args) {
    int i;                //1
    private int a = 1;     //2
    protected int b = 1;   //3
}
```

```

    public int c = 1;           //4
    System.out.println(a+b+c); //5
}

```

- 1.compiletime error at lines 1,2,3,4,5
- 2.compiletime error at lines 2,3,4,5
- 3.compiletime error at lines 2,3,4
- 4.prints 3

Explanation :

B is the correct answer.

The access modifiers public, protected and private, can not be applied to variables declared inside methods.

Question - 52

Which variables can an inner class access from the class which encapsulates it.

- 1.All final variables
- 2.All instance variables
- 3.Only final instance variables
- 4.All of the above

Explanation :

D is the correct answer.

Inner classes have access to all variables declared within the encapsulating class.

Question - 53

```

class c1
{
    public void m1()
    {
        System.out.println("m1 method in C1 class");
    }
}
class c2
{
    public c1 m1()
    {
        return new c1() {
            public void m1()
            {
                System.out.println("m1 mehod in anonymous class");
            }
        };
    }
}
public static void main(String a[])
{

```

```

c1 ob1 =new c2().m1();
        ob1.m1();
}}

```

- 1.prints m1 method in C1 class
- 2.prints m1 method in anonymous class
- 3.compile time error
- 4.Runtime error

Explanation :

B is the correct answer.

the anonymous class overrides the m1 method in c1.so according to the dynamic dispatch the m1 method in the anonymous is called

Question - 54

```

abstract class vehicle{
abstract public void speed();
}
class car extends vehicle{
public static void main (String args[]) {
vehicle ob1;
ob1=new car();    //1
}
}

```

- 1.compiletime error at line 1
- 2.forces the class car to be declared as abstract
- 3.Runtime Exception
- 4.None of the above

Explanation :

B is the correct answer.

Abstract method should be overridden otherwise Subclass should be abstract.

Question - 55

```

abstract class C1{
public void m1(){           //1
}
}
abstract class C2{
public void m2(){           //2
}
}

```

- 1.compile time error at line1
- 2.compile time error at line2
- 3.The code compiles fine

4.None of the above

Explanation :

C is the correct answer.

since the class C2 is abstract it can contain abstract methods

Question - 56

```
class base
{
base(int c)
{
    System.out.println("base");
}
}
class Super extends base
{
    Super()
    {
        System.out.println("super");
    }
public static void main(String [] a)
{
    base b1=new Super();
}
}
```

- 1.compile time error
- 2.runtime exception
- 3.the code compiles and runs fine
- 4.None of the above

Explanation :

A is the correct answer.

If super class has different constructor other than default then in the sub class you can't use default constructor.

Question - 57

```
class C1{
public void m1(){ // 1
}
}
class C2 extends C1{ //2
private void m1(){
}
}
```

- 1.compile time error at line1
- 2.compile time error at line2
- 3.Runtime exception
- 4.None of the above

Explanation :

B is the correct answer.

extending to assign weaker access not allowed. Overridden method should be more public.

Question - 58

```
class C1
{
    static interface I
    {
        static class C2
        {
        }
    }
}
public static void main(String a[])
{
    C1.I.C2 ob1=new C1.I.C2 () ;
    System.out.println("object created") ;
}
}
```

- 1.prints object created
- 2.Compile time error
- 3.Runtime Exception
- 4.None of the above

Explanation :

A is the correct answer.

A static interface or class can contain static members.Static members can be accessed without instantiating the particular class

Question - 59

```
class C1
{
    static class C2
    {
        static int i1;
    }
}
public static void main(String a[])
```



```

{
    System.out.println(C1.C2.i1);
}
}

```

1. prints 0
2. Compile time error
3. Runtime exception
4. None of the above

Explanation :

A is the correct answer.

static members can be accessed without instantiating the particular class

Question - 60

What will happen when you attempt to compile and run the following code

```

class Base{
protected int i = 99;
}
public class Ab{
private int i=1;
public static void main(String argv[]){
Ab a = new Ab();
a.hallow();
}

    abstract void hallow(){
        System.out.println("Claines "+i);
    }

}

```

1. Compile time error
2. Compilation and output of Claines 99
3. Compilation and output of Claines 1
4. Compilation and not output at runtime

Explanation :

A is the correct answer.

Abstract and native methods can't have a body: void hallow() abstract void hallow()

Question - 61

```

public class A extends Integer{
public static void main(Sring[] args){
System.out.println("Hello");
}
}

```

```
}  
}
```

What is the output?

- 1.Hello
- 2.Compile Error
- 3.Runtime
- 4.None of the above

Explanation :

B is the correct answer.

final class can't be extended. Integer is final class.

Question - 62

which statement is correct ?

- 1.A nested class is any class whose declaration occurs within the body of another class or interface
- 2.A top level class is a class that is not a nested class.
- 3.A top level class can be private
- 4.none of the above

Explanation :

A and B is the correct answer.

A nested class is any class whose declaration occurs within the body of another class or interface A top level class is a class that is not a nested class.

Question - 63

Constructor declarations can be include the access modifiers?

- 1.public
- 2.protected
- 3.private
- 4.All of the above

Explanation :

D is the correct answer.

constructor declarations may include the access modifiers public, protected, or private

Question - 64

```
private class B {

    public static void main(String[] args){
        System.out.println("DD");
        B b = new B();
    }

}
```

What is the output ?

- 1.DD
- 2.Compile Error.
- 3.Runtime Exception
- 4.None of the above.

Explanation :

B is the correct answer.

Only public, abstract and final is permitted for class modifier.

Question - 65

```
public class Point {
    int x = 1, y = 1;
    abstract void alert();
}
```

Is the code compile without error ?

- 1.compile without error
- 2.compile with error , because class should be abstract.
- 3.Can't say
- 4.None of the above.

Explanation :

B is the correct answer.

If there is any abstract method in a class then the class should be abstract.

Question - 66

```
abstract class Point {
    int x = 1, y = 1;
    abstract void alert();
}

public class A{
    public static void main(String[] args){
        Point p = new Point();
    }
}
```

```
}  
}
```

What is the output ?

- 1.compile without error
- 2.compile with error.
- 3.Can't say
- 4.None of the above.

Explanation :

B is the correct answer.

abstract class can't be instantiated.